University of Engineering and Technology, Abbottabad Campus

LAB 02 – FALL 2026

Class:	5th semester Software Engineering	Date:	25 Aug 2026
Subject:	Software Construction	Instructor:	Engr.Rizwan Shah
Time Allowed:	3 hr 00 min.	Lab Engineer:	

Semester Project Selection & UML Architecture

Objective: Before writing complex code, Software Engineers must thoroughly model the system to ensure correct architecture and behavior. In this lab, you will select your **Semester Project**. You will use this exact project for all future Software Construction labs throughout the rest of the semester.

By the end of this lab, you will have defined your project scope and translated its core logic into the four primary Unified Modeling Language (UML) diagrams: Use Case, Activity, Sequence, and Class.

Recommended Tooling: For this lab, you may use a digital modeling tool such as **draw.io**, **Lucidchart**, or **StarUML**.

Part 1: Semester Project Selection

Your Task: You must select a real-world software system to build this semester. Your project must be complex enough to require a backend system, object-oriented design, error handling, and file/database management.

Write 1-3 paragraph of **System Specification** detailing what your project is, who the users are, and what the core functionality will be. You will need to Approve this from me *before moving to Part 2.*

(For Reference Only): *Project Name: Automated Blueprint Extraction System Users will log into a web portal and upload vector PDF architectural plans. The system will validate the file format. If valid, the backend will process the document, sending images through an Object Detection model to find electrical symbols, and extracting text to build a lighting schedule table. Finally, the system will save the extracted data to a database and generate a downloadable Excel report.*

Part 2: Lab Tasks & UML Modeling

Using your newly approved Semester Project, complete the following four architectural diagrams.

Task 1: Use Case Diagram (The High-Level View)

Goal: Show *what* your system does and *who* interacts with it, without worrying about how the code works.

Instructions: Draw a Use Case Diagram for your project. You must include:

- **The System Boundary:** A large box representing your software.
- **Actors (Outside the box):** At least two distinct users or external systems interacting with your software.
- **Use Cases (Inside the box):** At least 4-5 core functionalities (e.g., Login, Upload, Process, Generate Report).
- **Relationships:** You must accurately use `<<include>>` (for mandatory sub-tasks, like validating data) and `<<extend>>` (for optional/conditional tasks, like displaying an error message).

Task 2: Activity Diagram (The Logical Flow)

Goal: Map the step-by-step logical execution of the *single most important process* in your project.

Instructions: Draw an Activity Diagram detailing the core workflow of your software. You must include:

- **Start and End Nodes:** Solid black circles and target circles.
- **Activities (Rounded rectangles):** Representing the specific actions your system takes.
- **Decision Nodes (Diamonds):** You must include at least one point of failure or choice (e.g., *"Is the input valid?"* or *"Did the database connect?"*). Show the "abnormal case" branching off and handling the error gracefully.
- **Fork and Join Nodes (Heavy black bars):** Show at least two tasks happening in parallel, joining back together before the process ends.

Task 3: Sequence Diagram (Chronological Interaction)

Goal: Show how specific objects inside your software talk to each other over time.

Instructions: Draw a Sequence Diagram for a successful user transaction.

- **Lifelines (Boxes at the top):** You must include a `User Interface` object, a `Controller/Manager` object, and a `Database/Storage` object. (You may add more if your system requires it).
- **Messages (Horizontal arrows):** Map the exact function calls passed between these objects from the start of the action to the final return message.
- **Activation Bars:** Use vertical rectangles on the lifelines to show when an object is actively executing logic.

Task 4: Class Diagram (The Structural Blueprint)

Goal: Design the actual Java architecture that you will begin coding in the next lab. Ensure High Cohesion and Low Coupling.

Instructions: Draw a Class Diagram for your backend system. You must design a minimum of **three interconnected classes**. For each class, you must include:

- **Class Name**
- **Attributes (Variables):** With exact data types and private visibility modifiers (`-`).
- **Operations (Methods):** With return types and public visibility modifiers (`+`).

Draw the relationship arrows (Associations/Dependencies) between these classes to show how they depend on each other.

Deliverables & Assessment

Export your four diagrams as `.PNG` or `.PDF` files. Compile them, along with your written System Specification from Part 1, into a single Lab Report document.